



ODTWARZANIE

Awarie w systemie

- procesu
 - przyczyny: zakleszczenie, timeout, błąd ochrony, niespójność
 - reakcje: abort, restart przetwarzania
- węzła
 - przyczyny: błędy programowe lub sprzętowe, błędy zasilania
 - reakcje: stop i restart ze zdefiniowanego stanu
- pamięci masowej
 - reakcje: rekonstrukcja z archiwum
- komunikacyjne
 - przyczyny: awaria medium lub urządzeń sieciowych
 - reakcje: naprawa, retransmisja



Typy awarii systemowych w modelu *fail-recovery*:

przerwa

system restartuje w tym samym stanie, który poprzedzał awarię

amnezja

system restartuje w predefiniowanym stanie niezależnym od stanu w momencie wystąpienia awarii

częściowa amnezja

system restartuje w stanie, którego część pokrywa się ze stanem w momencie wystąpienia awarii, a pozostała część jest predefiniowana (np. awarie, po których następuje restart serwera plików)



Odtwarzanie:

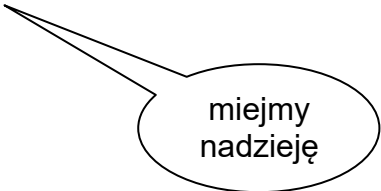
postępowe (*forward recovery*)

jeśli natura błędu powodującego awarię pozwala na usunięcie błędów ze stanu systemu i system (proces) jest wyposażony w mechanizmy obsługujące dany błąd, to błąd ten można efektywnie wyeliminować (naprawić) i umożliwić postęp przetwarzania

np. całkowita utrata precyzji w wyniku operacji zmiennopozycyjnej może zostać usunięta poprzez podanie wyniku „zero” w obsłudze błędu (*arithmetic underflow*)

wsteczne (*backward recovery*)

jeśli błąd jest nieprzewidywalny lub awaria nieodwracalna (np. *arithmetic overflow*), można jedynie wymienić cały stan systemu na wcześniej zarejestrowany, wolny od błędów



miejmy
nadzieję



ODTWARZANIE POSTĘPOWE

- zależy od skuteczności przewidywania i obsługiwanie błędów
- wymaga poprawnego przewidzenia konsekwencji błędów

„Trudno cokolwiek przewidzieć, a już szczególnie przyszłość”
– Niels Bohr

- projektowane jest pod konkretny system dla konkretnych błędów
- nie nadaje się do stosowania wobec błędów nieprzewidywalnych
- niemożliwe do zaimplementowania jako mechanizm systemowy



ODTWARZANIE WSTECZNE

- uniwersalne – nie zależy od rodzaju błędu
- stosowalne dla każdego systemu
- implementowalne jako ogólny mechanizm

Wymagania:

- wcześniejszy stan musi być poprawnie przywrócony oraz
- przywrócony stan musi poprzedzać wystąpienie uszkodzenia powodującego błąd

Cena:

- **narzut** – odtwarzanie wsteczne może wprowadzać duży narzut
- **nawrót** – na ogół nie ma gwarancji, że awaria nie wystąpi po odtworzeniu stanu
- **niepowtarzalność** (niemożliwość wycofania) – nie wszystkie komponenty systemu są odtwarzalne, a szczególnie interakcje zewnętrzne (np. wydane przez bankomat pieniądze)



ODTWARZANIE WSTECZNE

- uniwersalne – nie zależy od rodzaju błędu
- stosowalne dla każdego systemu
- implementowalne jako ogólny mechanizm

Wymagania:

- wcześniejszy stan musi być poprawnie przywrócony oraz
- przywrócony stan musi poprzedzać wystąpienie uszkodzenia powodującego błąd

*No one cares if you can back up,
only if you can recover!*

- niepowtarzalność (niemożliwość wycofania) – nie wszystkie komponenty systemu są odtwarzalne, a szczególnie interakcje zewnętrzne (np. wydane przez bankomat pieniądze)



ODTWARZANIE WSTECZNE

- uniwersalne – nie zależy od rodzaju błędu
- stosowalne dla każdego systemu
- implementowalne jako ogólny mechanizm

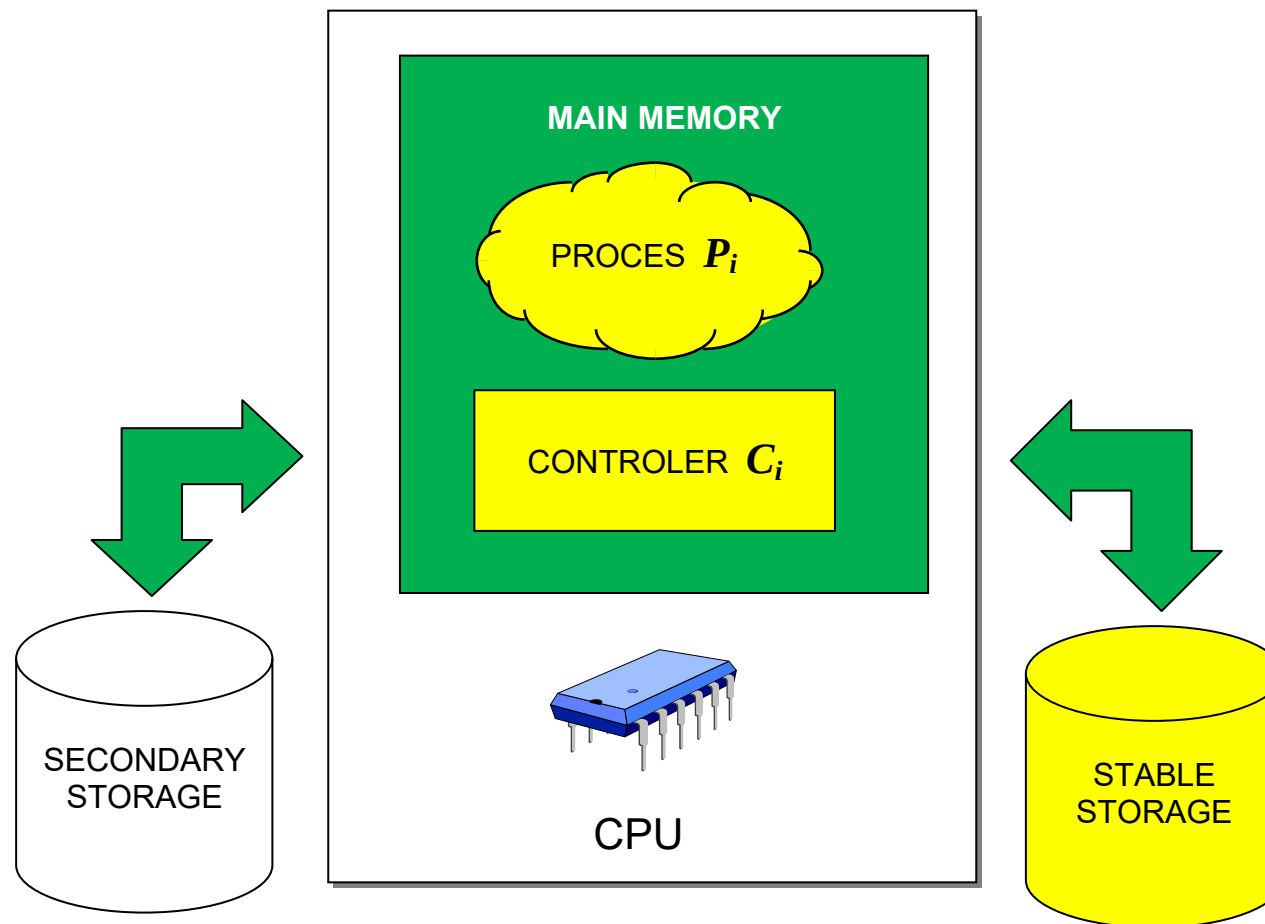
Wymagania:

- wcześniejszy stan musi być poprawnie przywrócony oraz
- przywrócony stan musi poprzedzać wystąpienie uszkodzenia powodującego błąd

Cena:

- **narzut** – odtwarzanie wsteczne może wprowadzać duży narzut
- **nawrót** – na ogół nie ma gwarancji, że awaria nie wystąpi po odtworzeniu stanu
- **niepowtarzalność** (niemożliwość wycofania) – nie wszystkie komponenty systemu są odtwarzalne, a szczególnie interakcje zewnętrzne (np. wydane przez bankomat pieniądze)

MODEL WĘZŁA SYSTEMU



→ NVM: ReRAM, 3DXpoint, ...



WSTECZNE ODTWARZANIE WĘZŁA

1. **Wycofywanie operacji** (operation-based recovery) – przechowywane w rejestrach (*log; audit trail*) niezbędne informacje o wykonywanych operacjach
 - strategie rejestrowania modyfikacji danych:
 - updating-in-place
 - write-ahead-log
2. **Przywracanie stanu** (state-based recovery) – przechowywany pełen stan procesu (pamięci, obiektu)
 - shadow pages
 - checkpointing
3. mix 1+2



Wycofywanie operacji:

Updating-in-place

- każdy zapis (*update*) rejestruje jednocześnie krotkę $\log = \langle \text{OBJ}, \text{UNDO}, \text{REDO} \rangle$, gdzie OBJ = identyfikator modyfikowanego obiektu, UNDO = stan obiektu sprzed modyfikacji, REDO = nowy stan obiektu

Odtwarzalna operacja jest implementowana w postaci kolekcji operacji:

- operacja *do*, wykonuje działanie (*update*) i rejestruje je (*log*)
- operacja *undo*, wycofuje działanie do zgodnie z polem UNDO
- operacja *redo*, odtwarza działanie do zgodnie z polem REDO

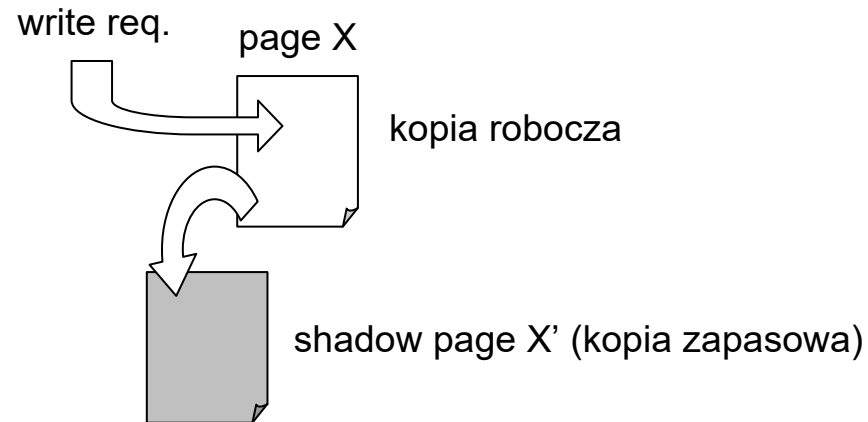
Problem braku atomowości: co jeśli awaria nastąpi pomiędzy dokonaniem zmian (*update*) a zapisaniem rejestru (*log*)?

Write-ahead-log

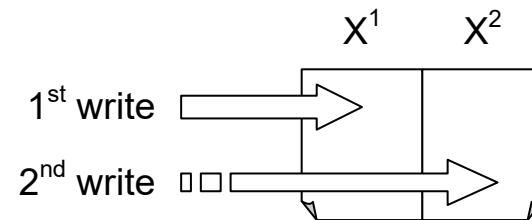
- *update* jest wykonywane zawsze po zapisaniu informacji UNDO
- przed zatwierdzeniem zmian zapisuje się informacje REDO

Przywracanie stanu

1. Shadow pages:



1.a) twin-page:



2. Checkpointing



PUNKT KONTROLNY (*checkpoint*)

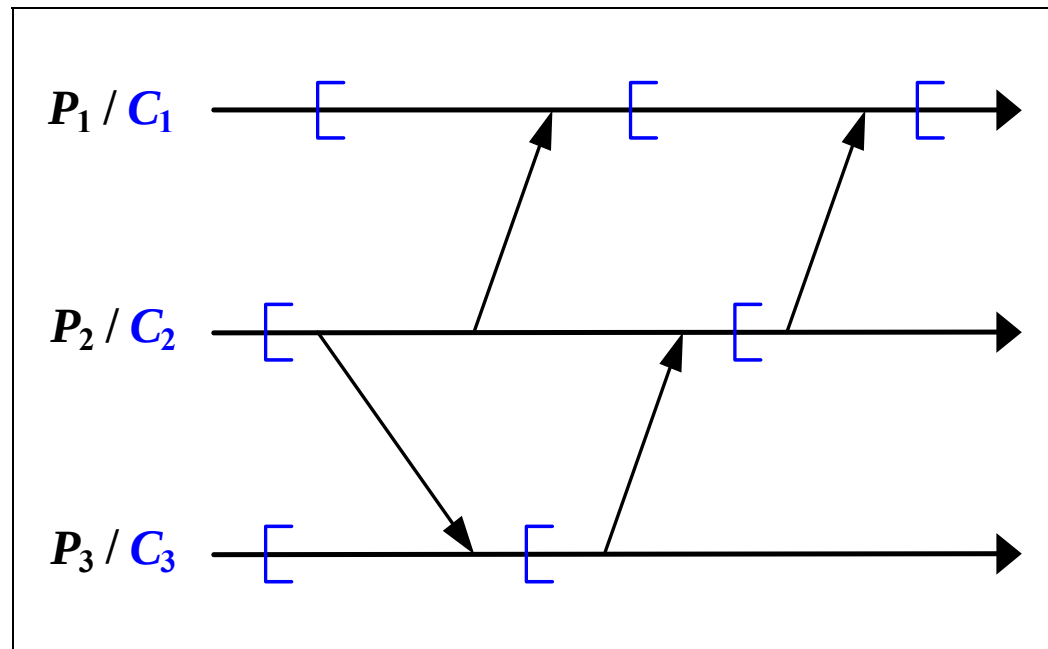
= **stan** należący do wykonania α , zachowany w pamięci trwałej w celu umożliwienia wznowienia przetwarzania od tego stanu w czasie odtwarzania wstecznego.

Lokalny punkt kontrolny

- punkt kontrolny cp_i procesu P_i (utworzony przez jego kontroler C_i)

Globalny punkt kontrolny

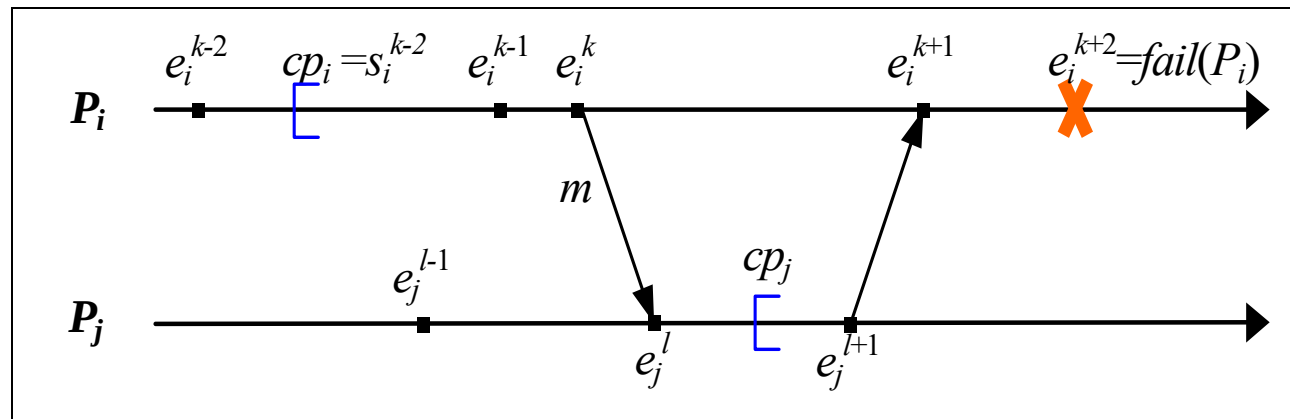
- wektor lokalnych punktów kontrolnych $CP = \langle cp_i: \text{dla każdego } P_i \rangle$





PARA PUNKTÓW KONTROLNYCH

= fragment globalnego punktu kontrolnego złożony z 2 lokalnych punktów kontrolnych

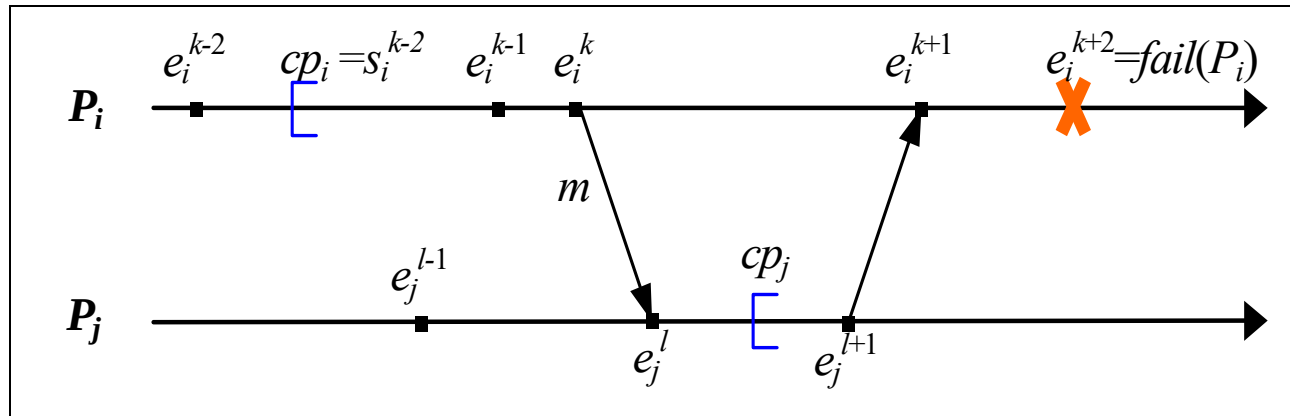


- para (cp_i, cp_j) zawiera wiadomość **osieroconą** m :

$$e_j^l = \text{recv}(P_j, P_i, m) \in cp_j \wedge e_i^k = \text{send}(P_i, \{P_j\}, m) \notin cp_i$$

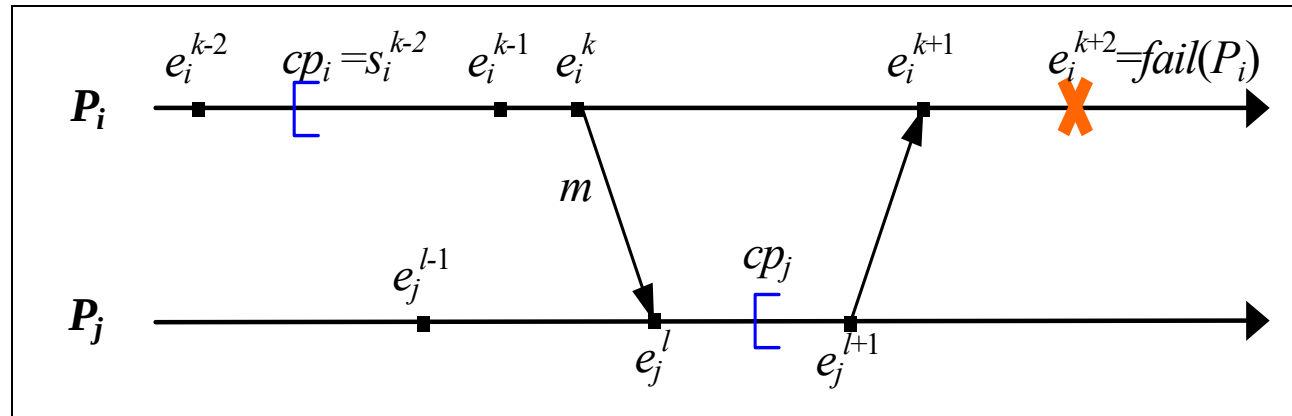
- czy taki niespójny obraz stanu przetwarzania jest akceptowalny dla odtwarzania?

Semantyka wiadomości osieroconych:



- w ogólnym przypadku **niespójny obraz stanu nie jest akceptowalny**
- może prowadzić do naruszenia własności bezpieczeństwa przetwarzania rozproszonego, np. jeśli osierocona wiadomość m przenosiła unikalny przywilej (żeton), po wznowieniu odtworzonego przetwarzania oba procesy posiadać będą przywilej;
- w istocie zatem, decydująca jest możliwość określenia czy osierocona wiadomość ma znaczenie dla poprawności przetwarzania, czy też nie (**znajomość semantyki wiadomości**)

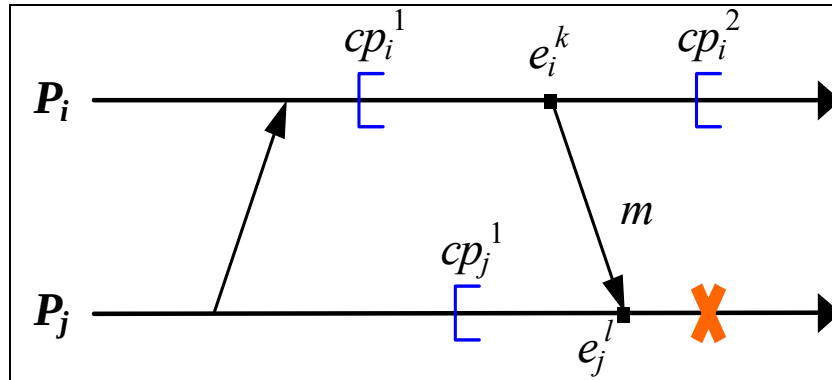
Determinizm / niedeterminizm:



- wznowienie przetwarzania od (cp_i, cp_j) wywoła ponownie zdarzenia e_i^{k-1} oraz e_j^{l+1} , o ile przetwarzanie jest deterministyczne
- jeśli zarówno e_i^{k-1} jak i e_i^k zachodzą deterministycznie, wówczas wiadomość m , już odebrana przez P_j (!), zostanie nadana ponownie → **duplikacja**
- wymagany jest zatem dodatkowy mechanizm tolerujący duplikację wiadomości
- gdy e_i^{k-1} lub e_i^k nie zachodzi deterministycznie, wynik lokalnego przetwarzania P_i jest nieprzewidywalny

Wiadomości utracone:

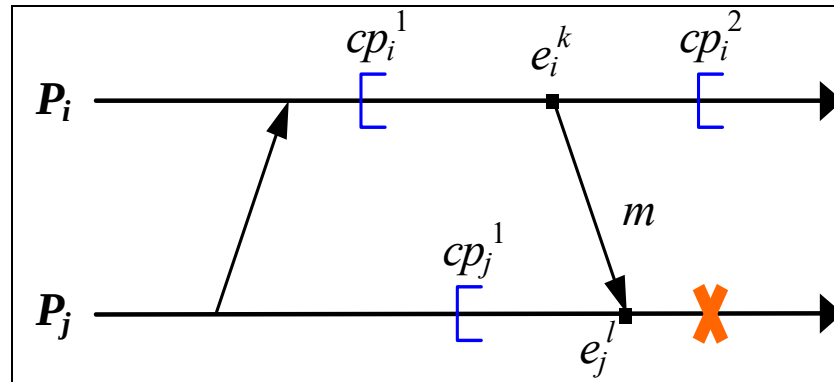
- w ogólnym przypadku **utrata wiadomości nie prowadzi do niespójności obrazu stanu**:



- para (cp_i^2, cp_j^1) zawiera wiadomość **utraconą** m :

$$e_i^k = \text{send}(P_i, \{P_j\}, m) \in cp_i^2 \wedge e_j^l = \text{recv}(P_j, P_i, m) \notin cp_j^1$$
- mimo to para ta (cp_i^2, cp_j^1) odpowiada spójnemu obrazowi stanu przetwarzania
- jednak dodatkowy mechanizm jest niezbędny dla odtworzenia/odzyskania (ang. *replay*) utraconych wiadomości znaczących (np. żetonu) $\rightarrow$ sliding window protocol

METODA KOLEJNYCH WYCOFAŃ:



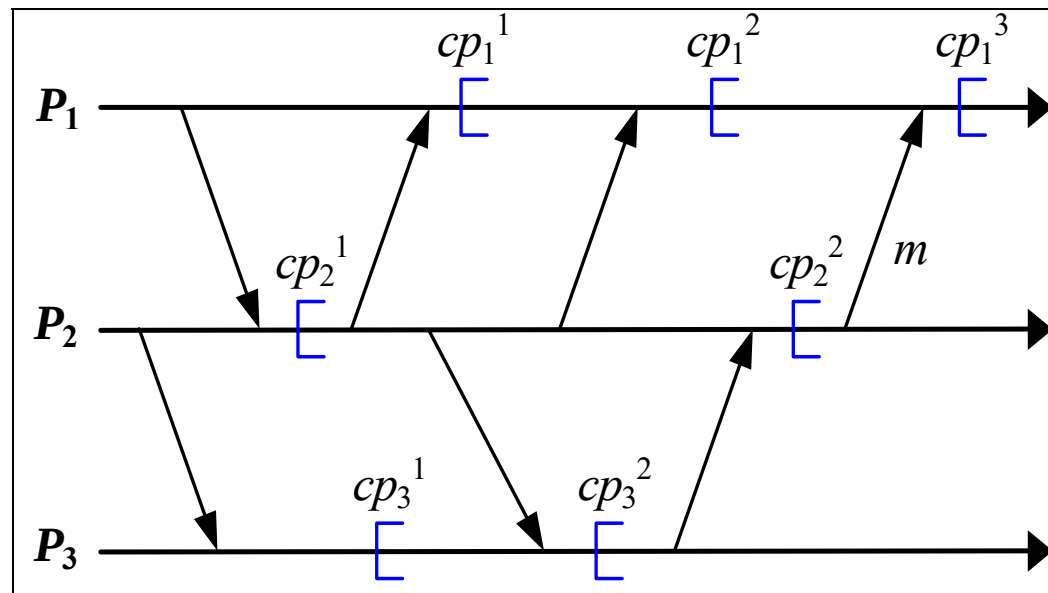
- zauważmy, iż zamiast wybrać do wznowienia przetwarzania globalny punkt kontrolny zawierający kontrowersyjną parę (cp_i^2, cp_j^1) alternatywnie można wycofać się do pewnego innego globalnego punktu kontrolnego zawierającego parę $(cp_i^{x<2}, cp_j^1)$, np. (cp_i^1, cp_j^1)
- podobnie możemy postąpić w przypadku wiadomości osieroconych
- oraz w dowolnym innym przypadku: gdy wybrany punkt kontrolny nie odpowiada obrazowi spójnemu (lub poprawnemu wg dowolnych innych kryteriów), możemy starać się wyszukać wcześniejszy punkt kontrolny, który spełni wymogi poprawnego odtworzenia przetwarzania



Efekt domino

Niech P_2 ulegnie awarii po wysłaniu wiadomości m .

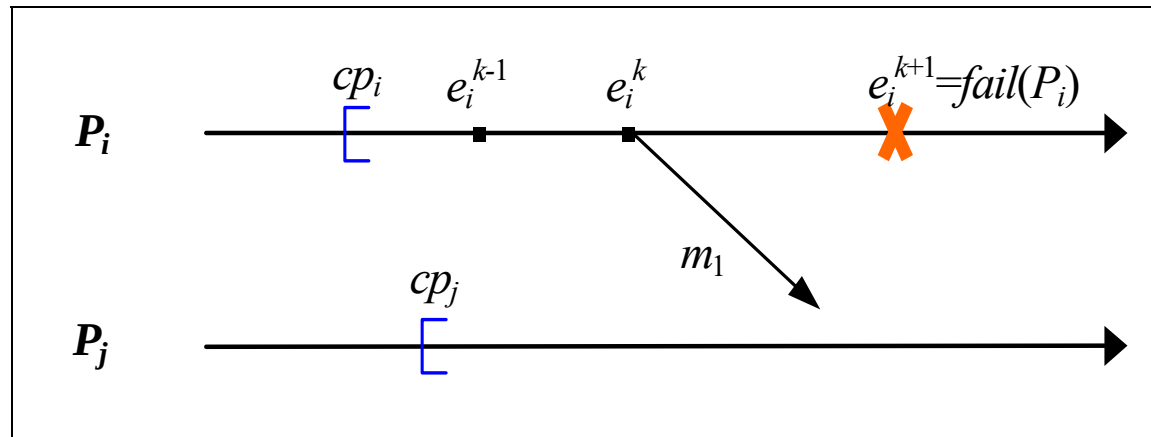
Po wycofaniu P_2 do cp_2^2 wiadomość m zostaje osierocona.



Dokąd wycofujemy stan przetwarzania metodą kolejnych wycofań?

A teraz, niech P_3 zostanie wycofany do cp_3^2 .

Niespójność stanu kanałów:



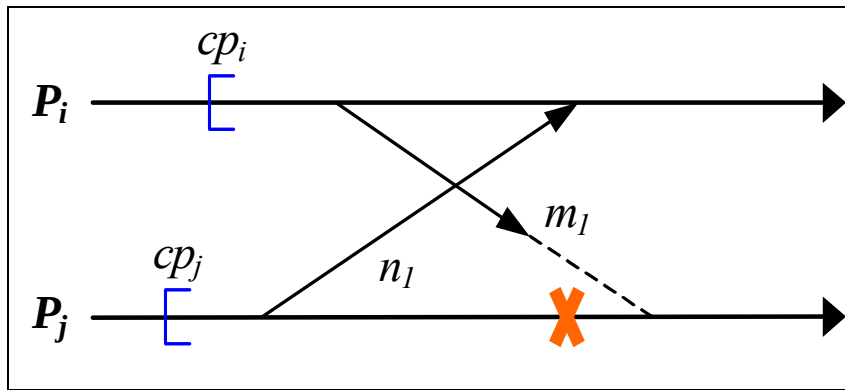
- jeśli oba zdarzenia e_i^{k-1} i e_i^k zachodzą deterministycznie, wiadomość m_1 zostanie nadana ponownie → **duplikacja**
- gdy e_i^{k-1} lub e_i^k nie zachodzi deterministycznie, po wznowieniu przetwarzania może zostać wysłana w jej miejsce całkowicie inna wiadomość m_2 , wówczas m_1 znajdująca się ciągle w kanale stanie się **osieroconą** w momencie odebrania, naruszając poprawność wznowionego przetwarzania



Ciągły restart:

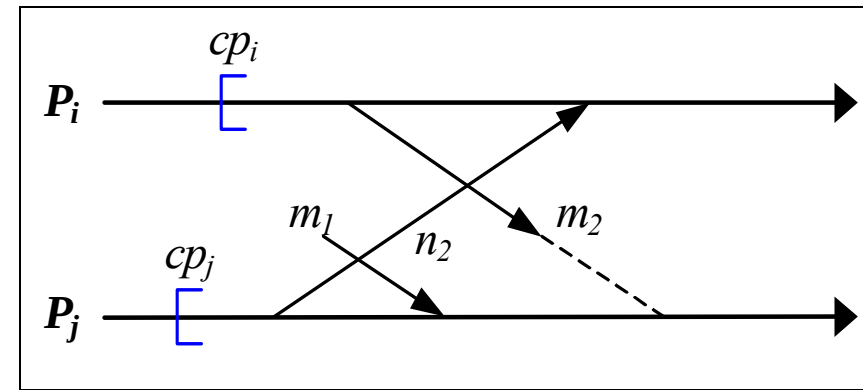
- ... prowadzić to może do problemu tzw. ciągłego restartu (*recovery livelock*):

przed restartem:



...

po restarcie:





SPÓJNOŚĆ

Spójność pary punktów kontrolnych

zależy od wielu czynników:

- determinizmu zdarzeń / przetwarzania
- semantyki wiadomości
- dostępności dodatkowych mechanizmów (obsługujących duplikację lub utratę wiadomości, niespójności stanu kanałów, itp.)

Spójność globalnego punktu kontrolnego

CPP = zbiór wszystkich par punktów kontrolnych

CCPP = zbiór wszystkich spójnych par punktów kontrolnych w CPP; $CCPP \subseteq CPP$

$CP = \langle cp_i: 1 \leq i \leq n \rangle$ jest spójnym punktem kontrolnym $\Leftrightarrow \forall_{1 \leq i, j \leq n} (cp_i, cp_j) \in CCPP$



Dalej przyjmujemy definicje odpowiadające spójnemu obrazowi stanu przetwarzania rozproszonego

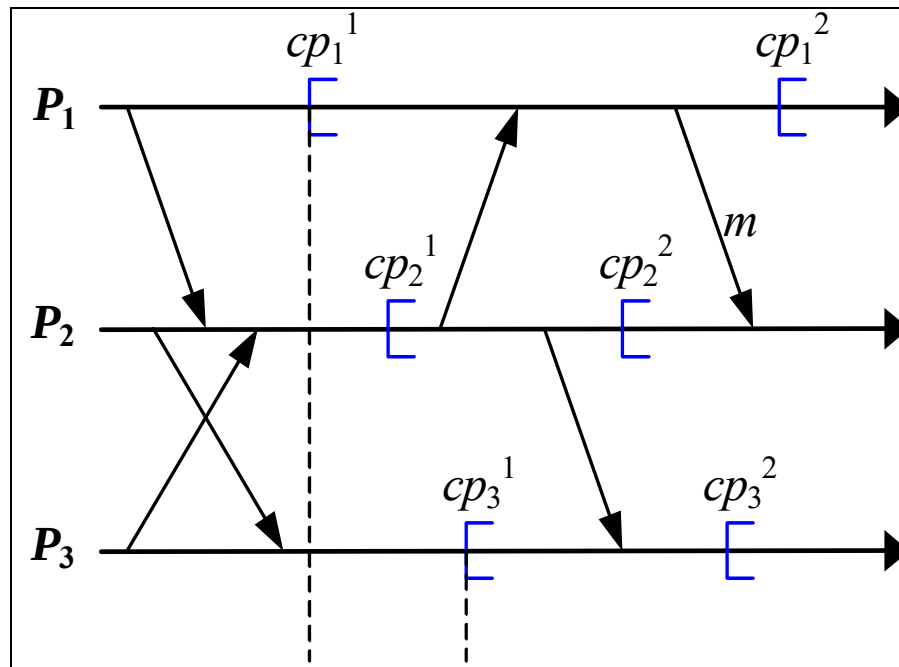
Spójny punkt kontrolny

$$CP^{\bullet} : (e \in CP \wedge e' \rightarrow e) \Rightarrow e' \in CP$$

$$(w\ szcz.: \text{recv}(m) \in CP^{\bullet} \Rightarrow \text{send}(m) \in CP)$$

Silnie spójny punkt kontrolny

$$CP^! : CP^{\bullet} + \text{send}(m) \in CP \Rightarrow \text{recv}(m) \in CP$$



$$CP^{\bullet} = \langle cp_1^2, cp_2^2, cp_3^2 \rangle$$

$$CP^! = \langle cp_1^1, cp_2^1, cp_3^1 \rangle$$



Proste ustanawianie CP' dla operacji atomowych

Ogólna idea:

- każdy proces P_i wyznacza punkt kontrolny cp_i bezpośrednio po wysłaniu kolejnej wiadomości

Założenie:

- operacje *send* i *checkpoint* są nierozdzielne (atomowe)

Redukcja kosztu:

- cp_i wyznaczany jest po wysłaniu $K > 1$ wiadomości
- jaki problem tu występuje?



LINIA ODTWARZANIA

GC = zbiór wszystkich globalnych punktów kontrolnych

GC[•] = zbiór wszystkich spójnych globalnych punktów kontrolnych; $GC^{\bullet} \subseteq GC$

Linia odtwarzania (*recovery line*):

= spójny globalny punkt kontrolny:

$$RL = \langle cp_1, cp_2, \dots, cp_n \rangle \in GC^{\bullet}$$

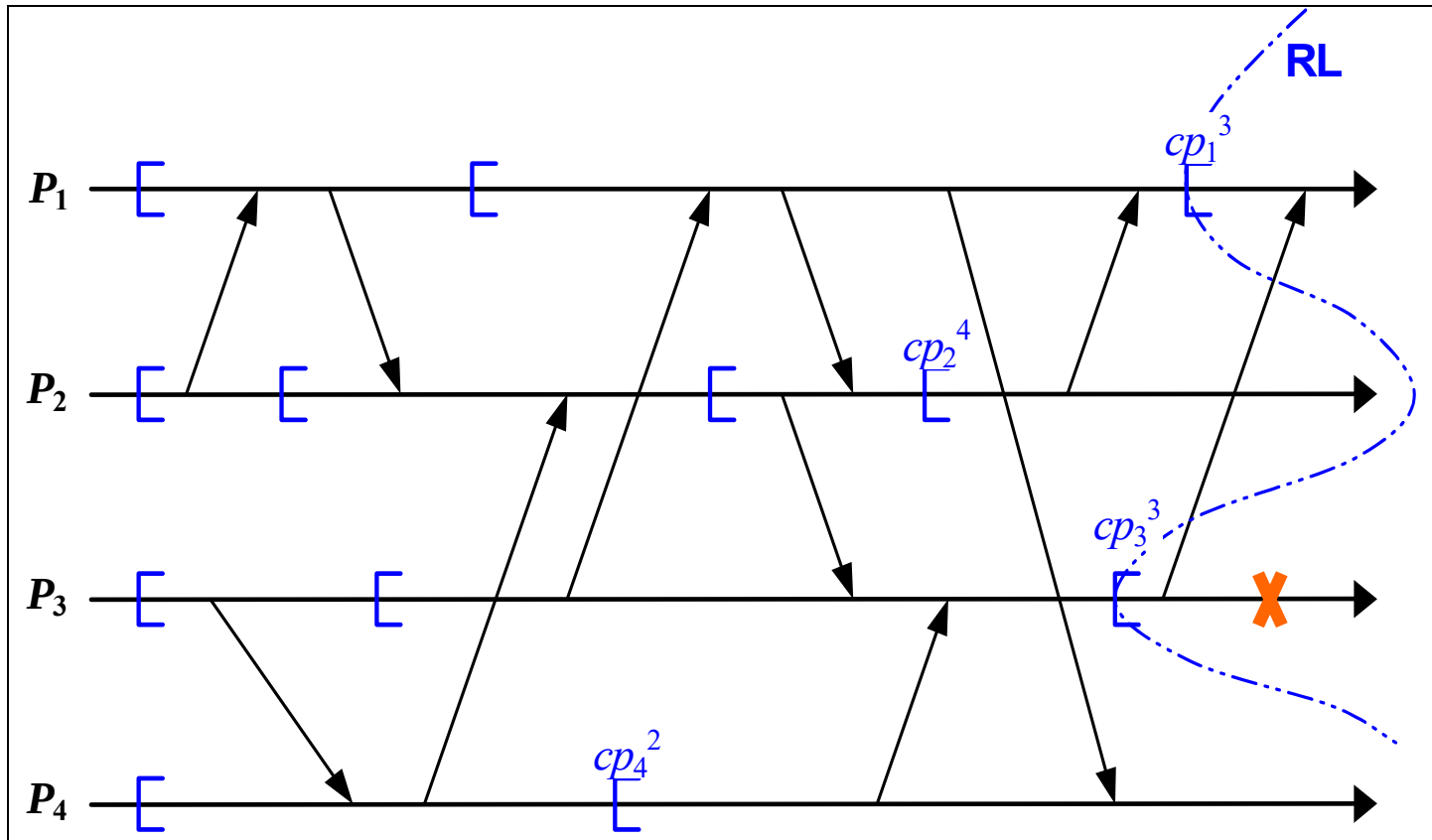
Punkt odtwarzania (*recovery point*)

= cp_i należący do jakiejś linii odtwarzania

Najświeższa linia odtwarzania (*current recovery line*):

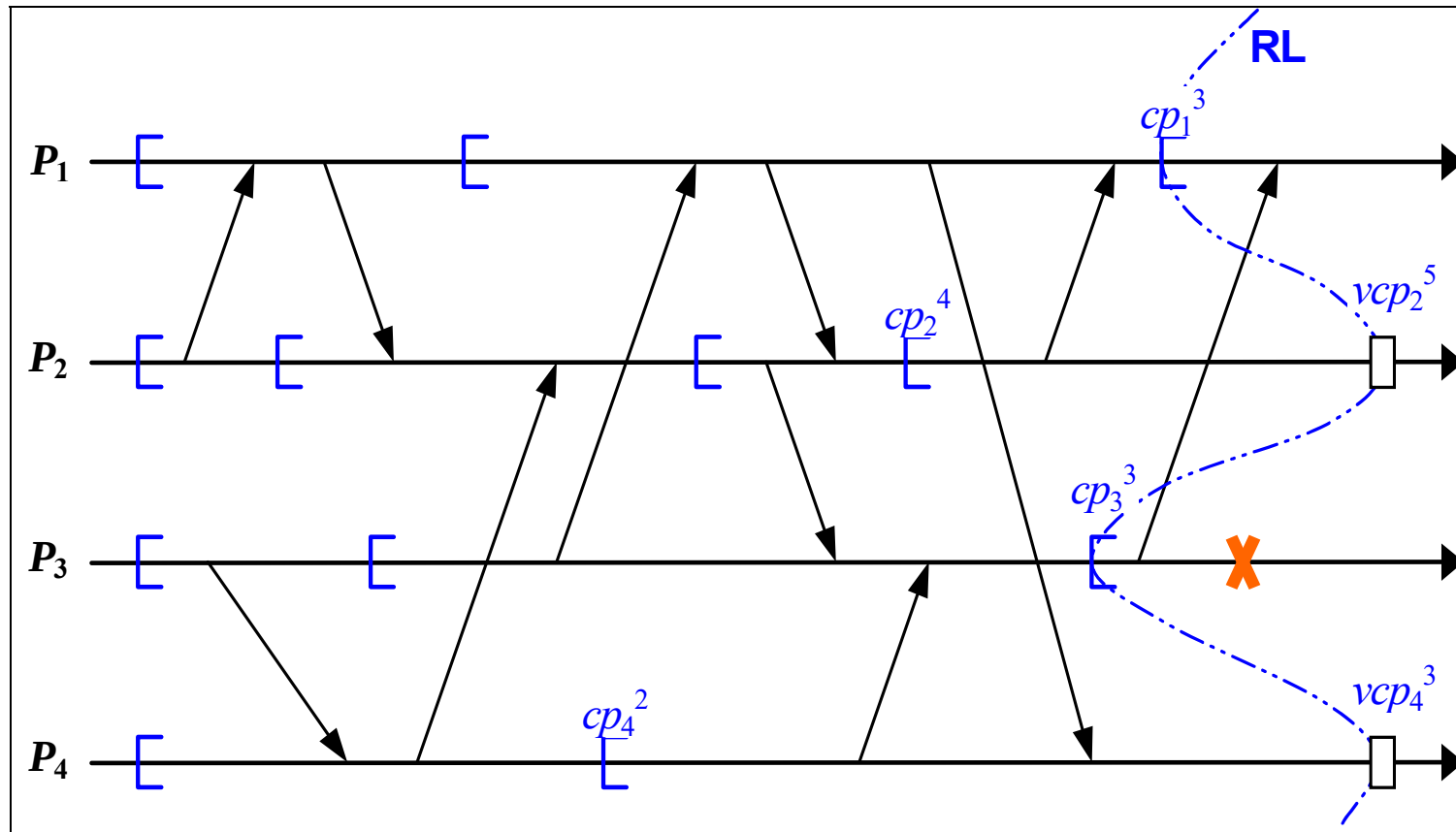
$$RL^* :: \bigvee_{RL \in GC^{\bullet}} RL \subseteq RL^*$$

Zawężenie linii odtwarzania



Po awarii P_3 linia odtwarzania $\mathbf{RL}=\{cp_1^3, cp_3^3\}$!

Wirtualne punkty odtwarzania



P_2 i P_4 uznają aktualny stan za *virtual checkpoint* (vcp_2^5 oraz vcp_4^3)

wirtualna linia odtwarzania: $\mathbf{RL} = \langle cp_1^3, vcp_2^5, cp_3^3, vcp_4^3 \rangle$



OUTPUT COMMIT

Problem interakcji ze światem zewnętrznym

- protokół odtwarzania nie wycofa operacji wykonanych na elementach spoza systemu

Algorytm *output commit*

- ma na celu zagwarantować, że stan, w którym wykonano takie operacje nigdy nie będzie musiał być wycofany
- może powodować dodatkowy narzut (blokowanie przetwarzania) przed każdą interakcją zewnętrzną
- ponadto, dane pobrane z zewnątrz muszą być składowane w pamięci trwałej, aby służyć do odtworzenia (ang. *replay*)
- algorytm *output commit* musi wchodzić w skład każdego protokołu odtwarzania stanu systemu z interakcjami zewnętrznymi – w dalszych rozważaniach będziemy milcząco przyjmować jego istnienie



SKOORDYNOWANE (SYNCHRONICZNE) TWORZENIE PUNKTÓW KONTROLNYCH

Sposób ustanawiania CP:

- tak, aby zawsze był to CP[•] (w efekcie RL^{*})

Algorytm Koo-Touega [2]

- wykorzystuje ideę przetwarzania dyfuzyjnego i 2PC

Dwa rodzaje punktów kontrolnych:

- ostateczny (*permanent checkpoint*) – należący do CP[•]
(tylko taki może być punktem odtwarzania)
- wstępny (*tentative checkpoint*) – zanim stanie się *permanent*
- punkty kontrolne obu rodzajów zapisywane są w pamięci trwałej
- jednocześnie istnieje co najwyżej jeden punkt ostateczny dla każdego procesu



Model systemu:

- przetwarzanie może być niedeterministyczne
- model awarii *fail-recovery*: procesy podlegają tymczasowym awariom typu *zatrzymanie*; co więcej – pozostałe procesy w skończonym czasie dowiadują się o awariach
- niezawodne kanały FIFO, w szczególności:
 - sieć niepodzielna na rozłączne części
 - za retransmisję wiadomości odpowiada podsystem komunikacyjny
- ewentualne interakcje ze światem zewnętrznym obsługuje dodatkowy protokół *output commit*

Dodatkowe założenia prezentacji:

- tylko jeden proces rozpoczyna wykonanie algorytmu (inicjator)
- procesy nie ulegają awariom podczas wykonywania tego algorytmu



Algorytm:

- Faza 1: inicjator C_i (kontroler procesu P_i) ustala wstępny cp_i i rozsyła żądanie wyznaczenia cp_j pozostałym C_j
- Faza 2: jeśli C_i otrzymał wszystkie potwierdzenia pozytywne, rozsyła decyzję zamiany wstępnych punktów kontrolnych na ostateczne
- C_j nie wysyła żadnych wiadomości P_j dopóki nie otrzyma decyzji

Wykorzystywane struktury:

- $m.l$ (m label) – etykieta wiadomości m (monotoniczny licznik wysyłanych wiadomości)
minimalna etykieta = $\perp$; maksymalna = T
- $last_label_rcvd_i[j]$ przechowuje etykietę ostatniej wiadomości m wysłanej przez P_j i odebranej przez P_i od ostatniego punktu kontrolnego:

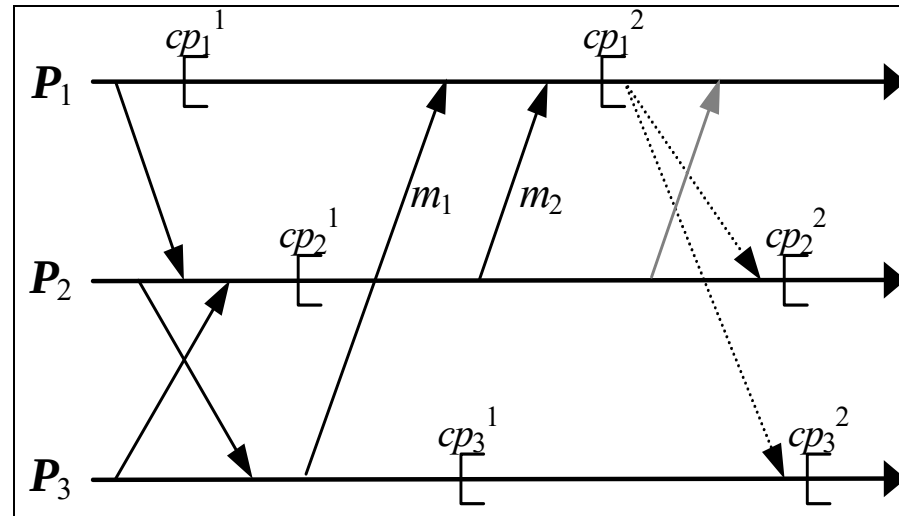
$$last_label_rcvd_i[j] = \begin{cases} m.l & \text{jeśli } m \text{ istnieje} \\ \perp & \end{cases}$$

- $first_label_sent_i[j]$ przechowuje etykietę pierwszej wiadomości m wysłanej przez P_i do P_j od ostatniego punktu kontrolnego:

$$first_label_sent_i[j] = \begin{cases} m.l & \text{jeśli } m \text{ istnieje} \\ \perp & \end{cases}$$



Niepotrzebne punkty kontrolne



$cp_3^2 ?$

Optymalizacja

Jeśli C_i wysyła do C_j żądanie $\text{Take_a_tentative_ckpt}(C_i, C_j, \text{last_label_rcvd}_i[j])$,
to C_j wyznacza wstępny punkt kontrolny procesu P_j tylko gdy:

$$\text{last_label_rcvd}_i[j] \geq \text{first_label_sent}_j[i] > \perp$$

Żądanie jest wysyłane tylko do $C_j \in \text{ckpt_cohort}_i = \{ C_j : \text{last_label_rcvd}_i[j] > \perp \}$



Odtwarzanie

Założenia:

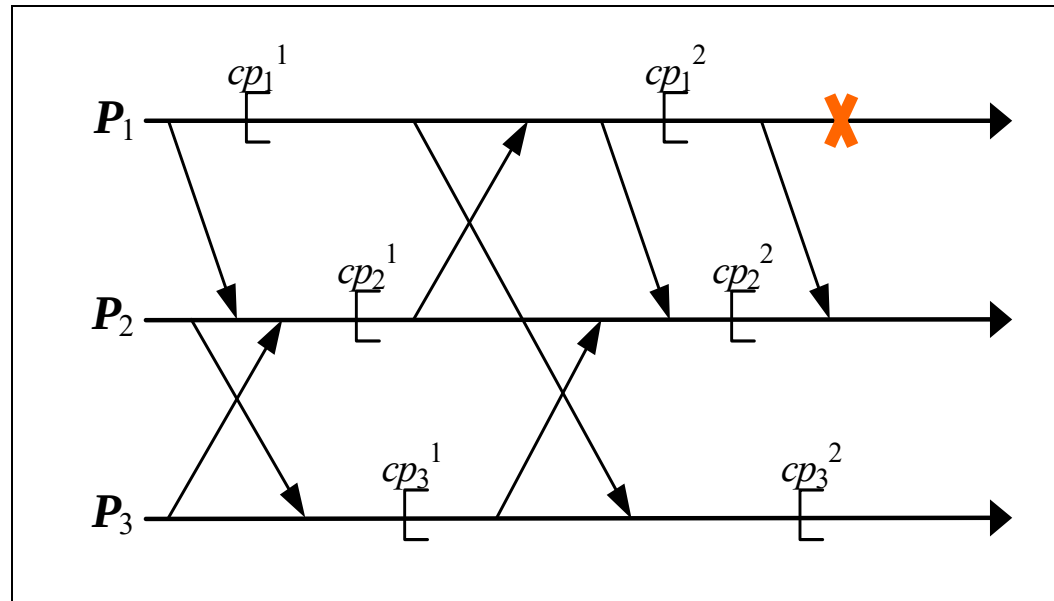
- tylko jeden inicjator
- procedury checkpointing i recovery nie wykonują się jednocześnie
- C_j nie zgadza się na wycofanie P_j , jeśli już uczestniczy w którejś z tych procedur

Faza 1: inicjator C_i wysyła pytanie o zgodę na wycofanie do poprzedniego cp_j

Faza 2: jeśli C_i otrzymał wszystkie potwierdzenia pozytywne, rozsyła decyzję wycofania do wszystkich procesów

C_j nie wysyła żadnych wiadomości dopóki nie otrzyma decyzji

Niepotrzebne wycofanie:

 $cp_3^2 ?$



Optymalizacja

- $\text{last_label_sent}_i[j]$ przechowuje etykietę ostatniej wiadomości m wysłanej przez P_i do P_j przed ostatnim punktem kontrolnym:

$$\text{last_label_sent}_i[j] = \begin{cases} m.l & \text{jeśli } m \text{ istnieje} \\ \top & \end{cases}$$

Jeśli C_i wysłała do C_j żądanie $\text{Prepare_to_rollback}(C_i, C_j, \text{last_label_sent}_i[j])$, to C_j wycofa P_j do ostatniego punktu kontrolnego tylko gdy:

$$\text{last_label_rcvd}_j[i] > \text{last_label_sent}_i[j]$$

Spełnienie tego warunku wskazuje, że P_i wycofuje się do stanu, w którym zostaną anulowane zdarzenia wysłania co najmniej jednej wiadomości odebranej już przez P_j .

- $\text{roll_cohort}_i = \{ C_j: P_i \text{ może wysyłać wiadomości do } P_j \}$



Wady skoordynowanego wyznaczania punktów kontrolnych:

1. Wymagane dodatkowe wiadomości kontrolne.
2. Synchronizacja może powodować duży narzut (opóźnienie).
3. Przy małej ilości awarii prowadzi do zbędnego ponoszenia dużego kosztu.
4. Koszt ponosimy nawet gdy w danym wykonaniu przetwarzania awarie w ogóle nie występują.



LITERATURA

- [1]  E.N. Elnozahy, D.B. Johnson, Y.M. Wang, “A Survey of Rollback-Recovery Protocols in Message-Passing Systems”, Carnegie Mellon University technical report CMU-CS-96-181, 1996.
- [2]  R. Koo, S. Toueg, “Checkpointing and Rollback-Recovery for Distributed Systems”, IEEE Transactions on Software Engineering, vol.13 no.1, pp.23-31, 1987.
- [3]  R. Chow, T. Johnson, "Distributed Operating Systems & Algorithms", Addison Wesley Longman, 1997, ch.13.
- [4]  M. Raynal, “Distributed algorithms for message-passing systems”, Springer, 2013, ch.8.